

Exam Prep – Spanish Poker Dice Platform

Quick-reference guide for the oral exam. One section per feature.

Stack: **React + Express.js + MongoDB + WebSockets (MERN)**

Table of Contents

1. Project Overview
 2. Authentication
 3. Home Page
 4. Lobby Page
 5. Create Game Page
 6. Game Page (playing)
 7. User Profile Page
 8. Tournament List Page
 9. Individual Tournament Page
 10. Admin Pages
 11. Comments (Real-time)
 12. Static Pages
 13. Header & Appearance Settings
 14. Backend Architecture
 15. Security
 16. Key Concepts
-

★ Study Priority Map

Headings throughout this doc are tagged with these. With 2 days, spend your time top-down.

● **Must know cold** · ● **Should know well** · ● **Know roughly** · ● **Don't memorize**

● **Highest priority — these are conceptual, can't be bluffed, and are the most-asked:** -

Reconciliation, Virtual DOM & the `key` role (Part 5.1) — *flagged in your Oblig 3 feedback* - `useEffect` **cleanup functions** (Part 5.2) — *flagged in your Oblig 3 feedback* - **Auth & token flow** — access vs refresh, HTTP-only cookies, IP-in-JWT, refresh-on-401 (§2, §15) - **Server-authoritative game + why WebSockets** — backend rolls dice so nobody cheats (§6) - **`req.mongoId`, not the request body** — the "post as someone else?" gotcha (§14, Part 3 #13) - **Core React fluency** — `useState`, `useEffect` deps, props vs state, Context - **Request flow + middleware order** + password hashing (scrypt + salt) (§14, §15)

● **Should know well:** `matchedData` / `errorHandler` -last · Mongoose schema-vs-model, `populate`, pagination, `pre` hooks · Web Components mechanics (`customElements.define`, lifecycle, `CustomEvent`) · ELO for 3–5 players + coin rules · the two security incidents · tournament round logic.

🟡 **Know roughly (don't drill):** exact numbers (8s polling, ± 200 ELO, 15-min token, 500 coins, 200 req/min) · home/lobby/profile page details · header & appearance.

🟠 **Don't waste time memorizing:** generic glossary terms you already know · version numbers, line counts, the Spanish hand-rank names verbatim · static pages.

The rule that beats all of this: you can be asked about *any* file, and AI-looking code triggers extra questions. The single highest-value prep is being able to **open your own code and explain it in your own words**. If a file feels unfamiliar when you read it, that's your real study list.

1. Project Overview 🟡

A multiplayer Spanish Poker Dice web app. Users register, join/create games, play in real-time, participate in tournaments, and earn ELO + coins.

Tech: - **Frontend:** React 19 + React Router, SCSS, Fetch API, WebSocket API, Web Audio API - **Backend:** Express.js + MongoDB/Mongoose, JWT, WebSockets (ws library), Nodemailer - **Key patterns:** JWT in HTTP-only cookies, role-based access, real-time gameplay via WebSocket, 18 fixed game variants

Folder structure: - backend/project/ — Express server - models/ — Mongoose schemas - controllers/ — handle requests, call services - services/ — business logic - routes/ — define API endpoints + validators - middleware/ — JWT auth, role checks, error handling - webSockets/ — real-time game engine - utils/ — JWT, hashing, hand evaluation, mailer, coins - frontend/src/ — React app - pages/ — one file per page - components/ — reusable UI pieces - contexts/ — global state (auth, appearance) - hooks/ — custom React hooks - api/ — all fetch calls to backend

2. Authentication 🟠

Pages: Login.jsx, Register.jsx, ForgotPassword.jsx, ResetPassword.jsx, VerifyEmail.jsx

Register flow

1. User fills in the form → POST /users
2. Backend creates user, generates a random token, sends **verification email**
3. User clicks link in email → goes to /verify-email?token=...
4. Frontend reads token from URL, sends POST /users/verify-email
5. Backend marks user as emailVerified: true
6. User can now log in

Login flow

1. POST /users/login → backend checks password, creates JWT access token (1h) + refresh token (30 days)
2. Both tokens set as **HTTP-only cookies** (JS can't read them — security)

3. Backend also stores the user's **IP** inside the JWT
4. Frontend stores user data in `sessionStorage` (cleared on tab close)
5. On every page refresh, `AuthContext` restores the user from `sessionStorage`

Token refresh

- When a request returns 401 (token expired), frontend automatically calls POST `/users/refresh`
- Backend checks the refresh token cookie, issues a new access token
- All other pending requests are queued until refresh is done (prevents race conditions)

Forgot / reset password

1. POST `/users/forgot-password` → backend sends email with reset link containing a token
2. User clicks link → `/reset-password?token=...`
3. Frontend extracts token, user types new password → POST `/users/reset-password`
4. Token expires after **15 minutes**

Context: `AuthContext.jsx`

- Holds `user`, `login`, `logout`, `register`, `updateUserData`
 - Polls every 30 seconds to detect if the user got banned
 - `useAuth()` hook used everywhere to access login state
-

3. Home Page 🟡

File: `pages/Home.jsx`

Shows: - **Hero section** — intro text + links - **Waiting games preview** — games that can be joined right now (polls every 8 seconds) - **Top ongoing games** — the most active games - **Upcoming tournaments** — 5 nearest upcoming tournaments, each clickable - **Platform activity** — number of active players, available games, recent games played

How it works: - Uses `useEffect` to fetch games, tournaments, and activity stats in parallel on mount - Uses `requestIdleCallback` to defer non-critical fetches until the browser is idle - Lobby games are polled every 8 seconds with `usePolling` - Games outside the user's ELO range are hidden (± 200 ELO), guests see everything

4. Lobby Page 🟡

File: `pages/Lobby.jsx`

Shows all games with status `waiting` — games the user can join.

Filtering: Users can filter by: - Number of rounds (3, 5, or 7) - Rules (straights allowed / not allowed) - Time control (10s, 30s, or 90s)

Pagination: Shows 6 games, "Load more" adds 6 more (backend pagination via `page` + `limit` query params).

Polling: Refreshes every 8 seconds to show new games without page reload.

Filtering logic (`useLobbyGames.js`): - Hides games the user already joined - Hides games where the user's ELO is too far from the desired opponent ELO (200 ELO gap) - Guests see all games

5. Create Game Page 🟡

File: `pages/CreateGame.jsx`

Form to configure and create a new match.

Settings the user picks: - Number of rounds (3/5/7) - Rules (straights allowed / not) - Time control (10s / 30s / 90s) - Number of players (2/3/5) - Optional: desired opponent ELO range - Optional: coin wager (buy-in)

How it works: 1. User fills out the form and submits 2. Frontend finds the matching `GameCategory` from the 18 fixed variants 3. POST `/matches` with the `gameCategoryId`, `maxPlayers`, `coinWager` 4. Backend creates the match with status `'waiting'`, locks the coin wager 5. Frontend navigates to `/game/:matchId`

18 game variants = 3 round options × 2 rule options × 3 time options

6. Game Page (playing) 🔴

File: `pages/Game.jsx` (~500 lines) + `webSockets/gameSocket.js` (~500 lines)

The most complex feature. Real-time multiplayer gameplay.

Connection

- On mount, opens a **WebSocket** connection to the backend
- Sends a `'join'` message with the `matchId`
- Backend verifies the JWT cookie, adds the socket to the game room
- If the player already has game state (rejoining), it is restored

Game phases

1. **Waiting** — players join. When the room is full, all must click "Ready"
2. **Rolling** — each player rolls dice, can hold some and re-roll others. Timer counts down
3. **Betting** — players fold, match, or raise the bet. Others' dice are hidden (bluffing!)
4. **Round end** — after 3 seconds (reveal delay), all dice are shown, winner of the round is determined
5. **Game end** — after all rounds, final standings, ELO updated, coins paid out

What the backend sends (via WebSocket)

- `roll-result` — your own dice (private)
- `player-rolled` — notification that someone rolled (no dice shown)
- `betting-start` — transition to betting phase
- `player-folded` / `player-bet` / `player-matched` — others' betting actions
- `round-end` — reveal all dice, announce round winner
- `game-end` — final standings, ELO deltas, coin changes

Dice and game board

- `dice-poker-board.js` and `dice-poker-die.js` are **Web Components** (not React)
- They are wrapped inside a React component on the game page
- Web Components handle the visual rendering of dice

Timing

- Each player has a **total time budget** (10/30/90 seconds) across all rolls in all rounds
- If a player runs out of time, dice are auto-rolled and they always match the bet
- If a player disconnects, they are auto-folded

After game ends

- Backend calls `recordMatch` in the service: updates ELO, pays out coins, saves result to DB
- Frontend refreshes user stats (coins, ELO) from the API
- If part of a tournament, the backend checks if all matches in the round are done

Comments on game page

- Comment sidebar shows existing comments, loaded on mount
- New comments appear in real-time via WebSocket (`broadcastMatchComment`)

7. User Profile Page 🟡

File: `pages/User.jsx`

Accessible at `/user/:userId` — shows any user's public profile.

Visible to everyone: - Username, profile image, about-me bio - ELO ratings (one per time control: 10s, 30s, 90s) - Total games played - Wins and losses in the last month - Earned trophies - Recent games (paginated, "Load more" button)

Visible only to the user themselves (or admins): - Email address

Editable by the user (on their own profile): - Username, email, about-me, profile image, password

Coins are also shown — users get 100 coins per week (granted lazily when the profile is fetched).

8. Tournament List Page 🟡

File: `pages/Tournament.jsx`

Accessible from the main menu. Shows all tournaments.

Two sections: 1. **Upcoming/Ongoing** — tournaments users can still join or watch 2. **Past tournaments** — finished ones (shown separately)

Each card shows: title, date/time, game variant, rounds, ELO range, buy-in, number of participants, status

Features: - **Pagination** — "Load more" button - **Sorting** — by date, title, or number of participants - **Search** — partial title match after typing 3+ characters (debounced) - Clicking a tournament goes to the individual tournament page

9. Individual Tournament Page 🟡

File: `pages/TournamentPage.jsx`

Shows full details of one tournament.

Content: - Full description, trophy image (full size) - Status: upcoming / ongoing / finished / cancelled - Date, ELO range, buy-in, game variant, number of rounds - Author (admin who created it) - **List of joined players** (if upcoming) - **Standings** (if ongoing/finished) — points and wins per player - **Current round's matches** — links to individual game pages - Countdown timer to next round - Comment section (real-time via WebSocket)

Join/Leave: - Logged-in users can join if the tournament is upcoming and they meet Elo/buy-in requirements - Users can leave at any time - Guests see a message or are redirected to login

Players in an active tournament: - Are automatically redirected to their game page when a match starts - Return to tournament page after the match ends

Admin controls (only visible to admins): - Delete tournament button - Cancel tournament button (users can still view but can't join) - Edit button → goes to admin tournament creation form, pre-filled with current values

How tournament rounds work

1. Admin or frontend calls PUT `/tournaments/:id/nextRound`
 2. Backend randomly pairs participants → creates Match objects for each pairing
 3. Players play their games (via normal game flow)
 4. When all matches in a round finish, next round can begin
 5. After the final round: winner is determined by points, trophy is awarded, 500 coins bonus
-

10. Admin Pages 🟡

All admin pages: no footer, minimal header (just logo + admin nav links).

Only accessible if `role === 'admin'`. Enforced by `AdminRoute` component on frontend and `requireAdmin` middleware on backend.

Menu: Dashboard → Users → Comments → Tournaments

Dashboard (`pages/admin/Dashboard.jsx`)

- New signups in the last 7 days
- Active matches (last 24h), active players (last week), available games right now
- Recent security incidents:
- **Rate limit hits:** Someone made >200 requests/minute — logged with their IP, user agent, timestamp
- **IP mismatches:** Someone's IP changed after login (possible session hijacking)

User Administration (`pages/admin/Users.jsx`)

- Paginated, searchable list of all users
- Shows: username, email, role, banned status, registration date
- Actions: **ban/unban** user, **change role** (user ↔ admin)

Comment Administration (`pages/admin/Comments.jsx`)

- List of recent comments (filterable)
- Shows: comment text, author, which match/tournament it belongs to
- Action: **delete** comment

Tournament Creation (`pages/admin/TournamentCreate.jsx`)

Form fields: - Title, description, date/time - Number of rounds - Game category (variant selector) - ELO min/max (optional skill gate) - Buy-in (coins required to enter) - Trophy: upload an image + title → creates a Trophy document

After creation: redirects to that tournament's page.

Tournament Edit (`pages/admin/TournamentEdit.jsx`):

Same form, pre-filled. Reachable from the "Edit" button on the tournament page.

11. Comments (Real-time) 🟡

Files: `components/CommentList.jsx`, `CommentItem.jsx`, `CommentForm.jsx`, `api/comments.js`

Comments exist on **game pages** and **tournament pages**.

How it works: 1. On page load: fetch existing comments via GET `/comments?targetId=...&targetType=match|tournament` 2. User submits comment → POST `/comments` 3. Backend saves comment, then broadcasts it to all WebSocket connections in that room 4. All other users on the same page **receive the new comment instantly** without refreshing 5. Admins see a delete button on every comment

12. Static Pages 🌐

Files: `pages/AboutUs.jsx`, `pages/AboutGame.jsx`, `pages/Privacy.jsx`, `pages/Terms.jsx`, `pages/Error404.jsx`

- About Us: team and platform info
 - About Spanish Poker Dice: game rules explanation
 - Privacy Policy: data handling
 - Terms and Conditions: usage rules
 - **404 page:** shown for any unrecognized route (catch-all `*` in `App.jsx`)
-

13. Header & Appearance Settings 🟡

Files: `components/Header.jsx`, `Navbar.jsx`, `Greeting.jsx`, `Appearance.jsx`

Header (`Header.jsx`)

Contains: - **Logo** — links back to homepage - **Navbar** — main navigation menu (also has login/logout links) - **Greeting** — welcome message + profile picture preview for logged-in users - **Appearance** — the settings panel (gear ⚙️ button) - **Hamburger button** — toggles the mobile menu (`menuOpen` state); on mobile, Greeting + Appearance move inside the Navbar

Appearance panel (`Appearance.jsx`)

An expandable dropdown (gear icon) for customizing the page. Required by the assignment.

Settings: - **Theme** — toggle light/dark mode (also resets the board color to the first color of the new palette) - **Board background color** — color swatches; the palette changes depending on the theme (light vs dark sets). Selected color gets a white outline - **Sound on/off** — toggles `soundEnabled` - **Lobby games count** — a range slider (1–15) controlling how many games show in the lobby preview - **Log out** button (only if logged in)

How it works: - Reads/writes preferences via `useAppearance()` from `AppearanceContext` - Preferences are saved to `localStorage` and synced to the backend profile if the user is logged in (debounced) - `useEffect` adds a click-outside + scroll listener to auto-close the panel - `useRef` (`containerRef`) is used to detect clicks outside the panel

14. Backend Architecture

Request flow

Request → CORS → Rate Limiter → setUserRole() (JWT decode) → Route → Validator → Controller -

Key middleware

- **setUserRole** — runs on every request. Decodes JWT from the HTTP-only cookie. Attaches `req.userRole`, `req.userId`, `req.username` to the request object. Also checks if the request IP matches the IP stored in the JWT — if not, logs a security incident and returns 401.
- **requireUser** — blocks anonymous users (403)
- **requireAdmin** — blocks non-admins (403)
- **errorHandler** — central catch for all thrown errors. Returns JSON with status + code.

Routes → Controllers → Services

- **Routes** define the URL and HTTP method, attach validators and middleware
- **Controllers** extract the validated data, call the service, send the response
- **Services** contain all business logic (creating users, pairing tournament players, recording match results, etc.)

Models (MongoDB schemas)

Model	Purpose
User	accounts, ELO, coins, preferences, trophies, tokens
Match	one game session, players, result, ELO changes, coin wager
GameCategory	one of 18 fixed variants (rounds + rules + time)
Tournament	bracket, rounds, participants, status
Comment	comment on a match or tournament
Trophy	image + title awarded to tournament winners
Security	logs of rate-limit and IP-mismatch incidents

WebSocket (`webSockets/gameSocket.js`)

- Attached to the same HTTP server as Express
- Maintains **in-memory** game state (not persisted between server restarts)
- Each match has a "room" — a set of connected WebSocket clients
- On `'join'`: authenticate via JWT, add to room, restore state if rejoining
- Handles all rolling, holding, and betting message exchanges
- Calls the `recordMatch` service when the game ends to persist the result

15. Security

Feature	How
JWT in HTTP-only cookies	JS cannot read the token → XSS-proof
IP stored in JWT	Any request from a different IP → 401 + security log
Rate limiting	200 req/min per IP → 429 + security log
Password hashing	<code>bcrypt</code> with a unique random salt per user
Email verification	Users must verify email before playing
Token expiry	Access: 1 hour. Refresh: 30 days
CORS	Only specific frontend origins allowed
Role checks	<code>requireUser</code> / <code>requireAdmin</code> on every protected route
Input validation	<code>express-validator</code> on all POST/PUT endpoints

16. Key Concepts

ELO Rating

- All users start at **1000 ELO**
- Separate ELO for each time control (10s, 30s, 90s)
- Calculated after every game using `calculateEloDeltas` in `utils/handEvaluator.js`
- With multiple players: ELO is calculated as if every pair played each other (win vs those with fewer final points, lose vs those with more)

Coin System

- **100 coins/week** granted lazily — when a user's profile is fetched, the backend checks if 7 days have passed since the last grant
- Coins are wagered on games: the winner takes the pot
- **500 coins** bonus for winning a tournament
- Users need enough coins for the buy-in to join a game or tournament
- Coins are locked when a match starts; paid out (or returned if abandoned) at the end

Hand Evaluation (`utils/handEvaluator.js`)

Rankings (best → worst): 1. Repóker — 5 of a kind 2. Póker — 4 of a kind 3. Full — 3 + 2 4. Escalera — straight (78JQK or 8JQKA), only if rules allow 5. Trío — 3 of a kind 6. Doble Pareja — two pairs 7. Pareja — one pair 8. Carta Alta — high card

useEffect patterns used

- Fetch data on mount: `useEffect(() => { fetchData() }, [id])`
- Polling: `useEffect` sets up an interval, clears it on unmount
- WebSocket: connect on mount (`new WebSocket(...)`), send messages, add listeners, close on unmount
- Dependency arrays control when effects re-run

useContext / contexts

- `AuthContext` → who is logged in (read with `useAuth()`)
- `AppearanceContext` → theme, sound, colors (read with `useAppearance()`)
- Both wrap the entire app in `main.jsx`

Custom hooks

- `useFetch(fetchFn, deps)` — generic loading/error/data hook, cancels on unmount
- `usePolling(callback, interval, enabled)` — repeats a fetch every N ms
- `useSoundEffects()` — play/stop sounds (Web Audio API, respects `soundEnabled`)
- `useDebounceValue(value, delay)` — wait for user to stop typing before triggering search

Web Components

- `dice-poker-board.js` and `dice-poker-die.js` — custom HTML elements (not React)
- Registered with `customElements.define()`
- Used inside a React wrapper component with a `ref` to pass data in
- Separates the visual dice rendering from the React game logic

App.jsx routing

- All routes are defined here with `<Route path="..." element={...} />`
- `React.lazy()` + `<Suspense>` for code splitting (pages load on demand)
- `<AdminRoute>` wraps all `/admin/*` routes (redirects non-admins)
- `<Layout>` wraps normal pages (renders Header + Footer)
- Admin pages skip the Layout (no footer, minimal header)

Part 2 – Concept & Terminology Glossary

Every term, function, hook, and pattern used in the project, explained in general.
If you can explain each of these in one sentence, you can talk about any file.

Contents

- A. JavaScript / ES Concepts
 - B. React Core
 - C. React Hooks
 - D. React Router
 - E. Our Custom Hooks
 - F. Browser / Web APIs
 - G. Backend – Express
 - H. Backend – MongoDB / Mongoose
 - I. Authentication & Crypto
 - J. Libraries Used
 - K. General Patterns & Terms
-

A. JavaScript / ES Concepts

- **Module (import/export)** — Gives you separation of concerns and reusability by splitting code into multiple files. `export` makes something available; `import` pulls it in. We used ES modules (`"type": "module"`).
- **Arrow function** — Short function syntax: `() => {}`. Doesn't have its own `this`.
- **Async / await** — A way to write asynchronous code that reads like synchronous code. An `async` function always returns a Promise; `await` suspends *that function* until the awaited Promise settles, without blocking the rest of the program. Used for all DB queries and fetch calls.
- **Promise** — An object representing a value that will exist later (success or failure). `.then()` / `.catch()` or `await`.
- **Promise.all([...])** — Runs multiple promises in parallel and waits for all of them (used to fetch several things at once).
- **Destructuring** — Pulling values out of objects/arrays: `const { user } = useAuth()` or `const [a, b] = array`.
- **Spread / rest (...)** — Spread copies values into a new object/array (`{...obj, x: 1}`); rest collects them (`function(...args)`). Used a lot for immutable state updates.
- **Template literal** — Strings with backticks and `${variable}` interpolation.
- **Ternary operator** — Inline if/else: `condition ? a : b`. Used heavily in JSX for conditional rendering.
- **Optional chaining (?.)** — Safely access nested properties without crashing if something is undefined: `user?.profile?.name`.
- **Nullish coalescing (??)** — Fallback only if value is `null / undefined`: `theme ?? "dark"`.
- **Array methods** — `.map()` (transform each item → used to render lists), `.filter()` (keep some items), `.find()` (first match), `.reduce()` (combine into one value), `.sort()`.
- **JSON.stringify / JSON.parse** — Convert objects to/from text. Used for localStorage and WebSocket messages.

- **try / catch + throw** — Error handling. Wrap risky code in `try`, handle failures in `catch`. `throw new Error(...)` raises an error (backend throws custom errors with status codes that the error middleware catches).
 - **Closures** — A function "remembering" variables from where it was created (relevant to how hooks capture state).
-

B. React Core

- **Component** — A reusable function that returns JSX (UI). The whole app is built from components.
 - **JSX** — HTML-like syntax inside JavaScript (it is *not* HTML). The build tool compiles it into plain JS calls that create React elements — classically `React.createElement`, or the modern `jsx()` runtime, which is why React 19 no longer needs `import React`.
 - **Props** — Inputs passed into a component from its parent (read-only).
 - **State** — Data a component owns that can change over time; changing it re-renders the component.
 - **Re-render** — React re-runs a component function when its own state changes, its props change, or its parent re-renders. The re-run builds a new Virtual DOM, which React diffs against the old one (see Part 5.1) — it only touches the real DOM if something actually changed.
 - **Conditional rendering** — Showing different UI based on a condition (`{isLoggedIn && <Menu/>}` or a ternary).
 - **List rendering + key** — Using `.map()` to render arrays of elements. Each needs a unique **key** prop so React can track which item changed (we use IDs).
 - **Fragment (`<> </>`)** — Wrapper that groups elements without adding an extra DOM node.
 - **Controlled component** — A form input whose value is driven by React state (`value={x}` + `onChange`). All our forms work this way.
 - **Lifting state up** — Moving shared state to a common parent so siblings can use it.
 - **Context API / `createContext`** — Way to share state across the whole tree without passing props down every level. `createContext()` creates the context object; we use it for Auth and Appearance.
 - **Provider** — The component that supplies a context's value (`<AuthProvider>`). Wraps the app in `main.jsx`.
 - **`React.lazy` + `Suspense`** — Load a component only when needed (code splitting). `Suspense` shows a fallback (spinner) while it loads. Used for pages in `App.jsx`.
 - **ref** — A reference to a DOM element or a value that survives re-renders without causing one. Used to talk to Web Components and detect outside clicks.
 - **`createPortal`** (react-dom) — Renders a component into a different part of the DOM (outside the parent), used for modals/overlays like the ban notice and confirm dialogs so they sit on top of everything.
-

C. React Hooks

Hooks are special functions (their names start with `use`) that let function components use React features like state, side effects, refs, and context.

- **useState** — Adds a piece of state to a component. Returns `[value, setValue]`. (Most-used hook in the project.)
- **useEffect** — Runs side effects (things outside rendering): fetching data, timers, WebSocket connections, event listeners. Runs after render. The **dependency array** controls when it re-runs; the **return function** is cleanup (runs on unmount or before the next run). Example use: open a WebSocket on mount, close it on unmount.
- **useRef** — Holds a mutable value or a DOM reference that persists across renders but doesn't trigger re-renders. Used for DOM nodes (Web Components) and "outside click" detection.
- **useContext** — Reads a value from a Context provider (e.g. the logged-in user).
- **useMemo** — Caches the result of an expensive calculation so it isn't recomputed on every render (only when dependencies change).

Rules of Hooks: only call hooks at the top level of a component (never inside loops/conditions), and only from React functions.

D. React Router

Library for client-side navigation (changing pages without reloading).

- **<BrowserRouter>** / **Routes** / **Route** — Define which component shows for which URL path.
 - **<Link>** — Navigation that doesn't reload the page (instead of `<a href>`).
 - **<NavLink>** — Like **<Link>** but knows when it's the active route, so you can style the current page's menu item.
 - **<Navigate>** — A component that redirects when rendered (e.g. send non-admins away from `/admin`, or redirect after an action).
 - **Outlet** — Placeholder where a nested route renders (used in **Layout** to render the current page between Header and Footer).
 - **useNavigate** — Hook to navigate programmatically in code (e.g. after creating a game, go to the game page).
 - **useParams** — Read dynamic URL parts, e.g. `:id` in `/game/:id`.
 - **useLocation** — Read the current URL/location object (e.g. to know which page you're on).
 - **useSearchParams** — Read/write query string params (e.g. `?token=xyz` for verification, or `?page=2`).
-

E. Our Custom Hooks

We wrote reusable hooks (located in `hooks/`):

- **useFetch(fetchFn, deps)** — Generic data-fetching hook. Returns `{ data, loading, error }`, handles cancellation on unmount.
- **usePolling(callback, interval, enabled)** — Repeatedly calls a function every N milliseconds (used for live game/lobby updates). Cleans up the interval automatically.

- `useDebounceValue(value, delay)` — Delays updating a value until the user stops typing (used for search inputs so we don't query on every keystroke).
 - `useSoundEffects()` — Returns functions to play sounds (roll, hold, etc.) via Web Audio, respecting the sound setting.
 - `useLobbyGames.js` — Despite the filename, the export `filterLobbyMatches(matches, user)` is a plain helper function, **not** a React hook. It filters the lobby list (hides joined games and games outside the ELO range).
 - `useAuth()` — Custom context hook to read/modify the logged-in user.
 - `useAppearance()` — Custom context hook to read/modify theme, sound, and color preferences.
-

F. Browser / Web APIs

Built-in browser features (not React, not libraries):

- `WebSocket` — A persistent two-way connection between browser and server for real-time messages. Core of the live gameplay and live comments.
- `fetch` — Built-in function to make HTTP requests to the backend API.
- `AbortController` — Lets you cancel an in-flight fetch (we pass its `signal` so old requests are cancelled when components unmount or inputs change).
- `localStorage` — Persistent key-value storage in the browser (survives tab close). Used for appearance preferences.
- `sessionStorage` — Like `localStorage` but cleared when the tab closes. Used to keep the logged-in user across refreshes.
- `setTimeout` / `setInterval` — Run code once after a delay / repeatedly on an interval. Used for timers, polling, countdowns. Always cleared in cleanup.
- `addEventListener` — Listen for browser events (clicks, scroll). Used for "click outside to close" panels.
- `FormData` — Builds form/file payloads, used for image uploads (profile pictures, trophies).
- `URLSearchParams` — Build/read query strings (e.g. `?page=2&limit=6`).
- `requestIdleCallback` — Defer non-urgent work until the browser is idle (used on the homepage so important content loads first).
- **Web Audio API (AudioContext)** — Generate sounds in code (we synthesize click/roll sounds instead of loading audio files).
- **Web Components** — Native browser way to create custom reusable HTML elements (`<dice-poker-board>`, `<dice-poker-die>`). Required by the assignment. Key pieces:
 - `customElements.define('name', Class)` — Registers a custom element. The class extends `HTMLElement`.
 - `connectedCallback()` — Lifecycle method that runs when the element is added to the page (set up DOM, listeners).
 - `disconnectedCallback()` — Runs when the element is removed (cleanup).
 - `attributeChangedCallback()` — Runs when a watched attribute changes (re-render the dice).
 - `observedAttributes` — Static list telling the browser which attributes to watch for changes.

- **getAttribute / setAttribute** — Read/write attributes; how React passes data into the Web Component.
 - **Shadow DOM (attachShadow, shadowRoot)** — A private, encapsulated DOM + styles inside the element so its CSS doesn't leak in or out.
 - **CustomEvent + dispatchEvent** — How the Web Component sends data *back out* to React (e.g. "die clicked / hold this die"). React listens with `addEventListener`.
 - **document.createElement** — Create DOM nodes programmatically.
 - **querySelector** — Find an element in the DOM (used inside the Web Components and shadow root).
-

G. Backend – Express

Express.js is the web server framework.

- **Express app** — The server object that handles incoming HTTP requests.
 - **Route** — Maps an HTTP method + URL to a handler (`router.get('/users', ...)`).
 - **HTTP methods** — **GET** (read), **POST** (create), **PUT** (update), **DELETE** (remove).
 - **Middleware** — A function that runs in the request→response chain. Has access to `req`, `res`, `next`. Used for auth checks, validation, parsing, error handling. Calling `next()` passes control to the next middleware; `next(err)` jumps to the error handler.
 - **req / res** — The request object (incoming data: body, params, cookies) and response object (what you send back).
 - **Route params vs query vs body** — `/users/:id` (params), `?page=2` (query), JSON payload (body).
 - **express.json()** — Middleware that parses incoming JSON request bodies.
 - **Error-handling middleware** — Special middleware with 4 args (`err, req, res, next`) that catches thrown errors and sends a clean JSON error.
 - **Controller** — Function that handles a route: reads validated input, calls a service, sends a response.
 - **Service** — Holds the business logic (separated from controllers for clean architecture).
 - **REST API** — Design style where URLs represent resources and HTTP methods represent actions.
 - **Status codes** — 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests, 500 Server Error.
 - **CORS** — "Cross-Origin Resource Sharing." Browser security that controls which frontend origins may call the API. Configured to allow our frontend.
 - **Rate limiting** — Restricting how many requests an IP can make in a time window (prevents abuse). Logs incidents.
-

H. Backend – MongoDB / Mongoose

MongoDB = the database (stores JSON-like documents). Mongoose = library to model and query it.

- **Document** — One record (like a row), stored as a JSON-like object.
- **Collection** — A group of documents (like a table).

- **Schema** — Mongoose definition of a document's shape, fields, and types (`new Schema({...})`).
 - **Field options / validators** — Rules set on schema fields: `required` (must be present), `unique` (no duplicates, e.g. email), `default` (fallback value), `enum` (value must be one of a fixed list, e.g. status or game variant), `type`, and `ref` (link to another model).
 - **Model** — A constructor built from a schema; you use it to query/create documents (`User`, `Match`, etc.).
 - **ObjectId** — MongoDB's unique ID type for documents (`_id`).
 - **ref + populate()** — `ref` links a field to another model; `populate()` replaces the stored ID with the full referenced document (like a SQL join).
 - **Pre-hook (`schema.pre('save', ...)`)** — Code that runs before saving, e.g. hash the password or auto-generate an ID.
 - **index()** — Tells MongoDB to index a field for faster lookups.
 - **Query methods:**
 - `find()` — get many; `findOne()` / `findById()` — get one
 - `create()` / `save()` — insert
 - `findOneAndUpdate()` / `findByIdAndUpdate()` / `updateOne()` — update
 - `deleteOne()` / `deleteMany()` — remove
 - `countDocuments()` — count matches
 - `.select()` — Choose which fields to include/exclude (e.g. exclude the password).
 - `.sort()` / `.limit()` / `.skip()` — Order results, cap how many, and skip ahead — the basis of **pagination** (skip = (page - 1) × limit).
 - `.lean()` — Returns plain JS objects instead of full Mongoose documents (faster, read-only).
 - **Seeding** — Filling the DB with starter/dummy data via a script (`npm run seed`).
-

I. Authentication & Crypto

- **Authentication** — Verifying *who* the user is (login).
- **Authorization** — Verifying *what* they're allowed to do (roles: user/admin/anonymous).
- **JWT (JSON Web Token)** — A signed token holding user info. The server can verify it wasn't tampered with using a secret. `jwt.sign()` creates it, `jwt.verify()` checks it.
- **Access token** — Short-lived JWT (1h) proving you're logged in, sent with each request.
- **Refresh token** — Long-lived token (30d) used to get a new access token without re-logging in.
- **Cookie** — Small data stored by the browser and sent automatically with requests.
- **HTTP-only cookie (`httpOnly`)** — A cookie JavaScript can't read, protecting tokens from XSS attacks.
- **sameSite / secure** — Cookie flags controlling cross-site sending and HTTPS-only — protect against CSRF and interception.
- **Hashing** — One-way scrambling of passwords so the real password is never stored. We use `bcrypt` (a slow, secure hashing function).
- **Salt** — Random data added to each password before hashing so identical passwords get different hashes. Generated with `randomBytes`.
- **timingSafeEqual** — Compares two values in constant time to prevent timing attacks.

- **Email verification** — New users must confirm their email via a link with an expiring token before playing.
 - **XSS / CSRF** — Common web attacks (script injection / forged requests) that HTTP-only + sameSite cookies defend against.
 - **Security incident logging** — Recording suspicious events (rate-limit hits, IP mismatches) for the admin dashboard.
-

J. Libraries Used

Backend (package.json): - **express** — web server framework - **mongoose** — MongoDB modeling - **jsonwebtoken** — create/verify JWTs - **express-validator** — validate/sanitize request input. `validationResult()` collects errors; `matchedData()` returns only the validated fields. - **express-rate-limit** — request rate limiting - **multer** — handle file uploads (used in memory mode for images) - **nodemailer** — send emails (verification, password reset) - **cors** — enable cross-origin requests - **cookie-parser** — read cookies from requests - **ws** — WebSocket server (`WebSocketServer`) - **dotenv** / **--env-file** — load secrets from `.env`

Frontend (package.json): - **react / react-dom** — UI library (v19) - **react-router-dom** — client-side routing - **react-icons** — icon components - **sass** — SCSS preprocessor (variables, nesting, partials) - **vite** — dev server + bundler (fast builds, hot reload) - **eslint** — code linting

K. General Patterns & Terms

- **MERN stack** — MongoDB, Express, React, Node.js — the full JS stack we use.
- **Full-stack** — Working on both frontend (client) and backend (server).
- **Client-server model** — Browser (client) requests, server responds.
- **MVC-style separation** — Routes → Controllers → Services → Models. Keeps concerns separated.
- **SPA (Single Page Application)** — One HTML page; React swaps content via the router instead of full reloads.
- **Component-based architecture** — UI split into small reusable pieces.
- **Code splitting** — Loading only the code needed for the current page (lazy loading).
- **Pagination** — Loading data in pages/chunks instead of all at once ("load more" buttons).
- **Polling** — Repeatedly asking the server for fresh data on a timer.
- **Real-time / low-latency** — Pushing updates instantly via WebSocket instead of polling.
- **Debouncing** — Waiting until activity stops before acting (search inputs).
- **Optimistic vs server-authoritative** — Here the **server is authoritative**: the backend generates dice rolls and decides winners so players can't cheat.
- **Environment variables** — Config/secrets kept out of code in `.env` (DB URI, JWT secret, mail credentials).
- **Bundler / preprocessor / task runner** — Vite bundles JS; Sass preprocesses CSS; npm scripts run tasks (exam learning goal).

- **State restoration** — Rebuilding UI state after a reload (game state restored on rejoin, user restored from sessionStorage).
- **Validation** — Checking input is correct/safe before using it (frontend forms + backend express-validator).
- **Responsive design** — Layout adapts to screen size (e.g. the hamburger menu on mobile).

Part 3 – Practice Questions & Exercises

The teacher can ask you to **change or add code** on the spot. Section A is hands-on editing practice written in *your* code style. Section B is spoken/conceptual questions. Answers are hidden in collapsible blocks — try first, then click to check.

Section A – Coding Exercises (edit / add code)

A1. React / Frontend

Exercise 1. On the Lobby page, write a `useEffect` that logs `"Lobby mounted"` once when the page first loads.

Q: Solution

```
useEffect(() => {  
  console.log("Lobby mounted");  
}, []); // empty array = runs once on mount
```

The empty dependency array `[]` means it runs only on the first render.

Exercise 2. Add a piece of state called `searchTerm` (starting as an empty string) and an input that updates it.

Q: Solution

```
const [searchTerm, setSearchTerm] = useState("");  
// ...  
<input value={searchTerm} onChange={(e) => setSearchTerm(e.target.value)} />
```

This is a **controlled input** — its value comes from state.

Exercise 3. You have an array `tournaments`. Render a `<TournamentCard>` for each one.

Q: Solution

```
{tournaments.map(tournament => (
  <TournamentCard key={tournament.tournamentId} tournament={tournament} />
))}
```

Every list item needs a unique **key** — use the ID, never the array index when items can change.

Exercise 4. Show a `<Spinner />` while `loading` is true, otherwise show the content.

Q: Solution

```
if (loading) return <Spinner />;
// ...rest of the component renders the content
```

Or inline: `{loading ? <Spinner /> : <Content />}`.

Exercise 5. Make the Lobby poll for games **every 5 seconds** instead of 8.

Q: Solution

```
usePolling(fetchGames, 5000); // interval is in milliseconds
```

Exercise 6. Write a `useEffect` that fetches one tournament by `id` (from `useParams`) when the page loads and whenever `id` changes. Store it in state.

Q: Solution

```
const { id } = useParams();
const [tournament, setTournament] = useState(null);

useEffect(() => {
  getTournament(id)
    .then(data => setTournament(data))
    .catch(err => console.error(err));
}, [id]); // re-runs when id changes
```

Exercise 7. Only show a "Delete" button if the logged-in user is an admin.

Q: Solution

```
const { user } = useAuth();
// ...
{user?.role === "admin" && <Button onClick={handleDelete}>Delete</Button>}
```

`user?.` (optional chaining) avoids a crash if no one is logged in.

A2. Frontend API layer

Exercise 8. Add an API function `getTrophy(id)` that does a GET request to `/trophies/:id`.

Q: Solution

```
export async function getTrophy(id) {
  const res = await fetchWithAuth(`${BASE_URL}/trophies/${id}`);
  return handleResponse(res);
}
```

Matches the project pattern: `fetchWithAuth` then `handleResponse`.

Exercise 9. Write the API call that bans a user (PUT to `/users/:userId/ban`).

Q: Solution

```
export async function banUser(userId) {
  const res = await fetchWithAuth(`${BASE_URL}/users/${userId}/ban`, {
    method: "PUT"
  });
  return handleResponse(res);
}
```

A3. Backend – Routes, Controllers, Validators

Exercise 10. Add a route so **admins** can delete a tournament: `DELETE /tournaments/:tournamentId`.

Q: Solution

```
tournamentApiRouter.delete(
  '/tournaments/:tournamentId',
  requireAdmin,
  tournamentValidator.validateTournamentId(),
  validate,
  tournamentController.deleteTournament
);
```

Order matters: **role check** → **validator** → **validate** → **controller**.

Exercise 11. Write a controller function `getTrophy` that returns one trophy. Follow the project pattern (try/catch, `matchedData`, `next`).

Q: Solution

```
export async function getTrophy(req, res, next) {
  try {
    const { trophyId } = matchedData(req);
    const result = await trophyServices.getTrophy(trophyId);
    res.status(200).json(result);
  } catch (error) {
    next(error);
  }
}
```

`matchedData(req)` gives only the **validated** fields. Errors go to `next(error)` → the error handler.

Exercise 12. A new comment must not be empty. Add a validator chain for the `comment` body field that requires it.

Q: Solution

```
body("comment")
  .notEmpty()
  .withMessage("comment is required")
```

(Already in `validateCreateComment`.) Add `.isLength({ max: 1000 })` to also cap the length.

Exercise 13. In a controller, where does the user's ID come from when creating a comment — the request body or the token? Write the line.

Q: Solution

```
commentData.userId = req.mongoid; // from the verified JWT, NOT the body
```

Never trust a `userId` sent in the body — it could be faked. `setUserRole` middleware put `req.mongoid` there after verifying the token.

A4. Backend – Models & Mongoose

Exercise 14. Add a `pinned` boolean field to the Comment schema that defaults to `false`.

Q: Solution

```
pinned: {
  type: Boolean,
  default: false
}
```

Exercise 15. Write the Mongoose query to find all comments for one match (`targetId`), newest first.

Q: Solution

```
const comments = await Comment.find({ targetId, targetType: "match" })
  .sort({ createdAt: -1 }); // -1 = descending (newest first)
```

Exercise 16. Write the query to count how many matches currently have status `"ongoing"`.

Q: Solution

```
const count = await Match.countDocuments({ status: "ongoing" });
```

A5. WebSockets

Exercise 17. On the frontend, open a WebSocket and log every message received.

Q: Solution

```
const ws = new WebSocket("ws://localhost:3000");
ws.addEventListener("message", (event) => {
  const data = JSON.parse(event.data);
  console.log(data);
});
```

Messages arrive as text, so `JSON.parse` turns them back into objects.

Exercise 18. Send a `"join"` message with a `matchId` over the WebSocket.

Q: Solution

```
ws.send(JSON.stringify({ type: "join", matchId }));
```

You must `JSON.stringify` — WebSockets send strings, not objects.

Section B – Conceptual / Oral Questions

Likely "explain this" questions. Answers are short on purpose — say them out loud.

B1. React

Q: What is the difference between props and state?

Props are passed **in** from a parent and are read-only. State is **owned** by the component and can change; changing it re-renders the component.

Q: What does the dependency array in `useEffect` do?

It controls when the effect re-runs. `[]` = once on mount. `[id]` = whenever `id` changes. No array = after every render.

Q: Why does every item in a `.map()` list need a key?

So React can tell which items changed/added/removed between renders and update the DOM efficiently. We use a stable unique ID, not the index.

Q: What is the cleanup function in `useEffect` for?

The function you return from `useEffect`. It runs on unmount (and before the next effect run) to clean up — e.g. clear an interval, close a WebSocket, abort a fetch.

Q: What is Context and why do you use it here?

A way to share state across the whole component tree without passing props down every level. We use it for the logged-in user (`AuthContext`) and appearance settings (`AppearanceContext`).

Q: What does `React.lazy` + `Suspense` do in `App.jsx`?

Code splitting — each page is loaded only when its route is visited, instead of all at once. `Suspense` shows a Spinner while the page's code loads.

B2. Backend / Express

Q: What is middleware?

A function in the request→response chain with access to `req`, `res`, `next`. Used for auth checks, validation, parsing. `next()` passes to the next middleware; `next(err)` jumps to the error handler.

Q: Why split into routes, controllers, and services?

Separation of concerns. Routes define URLs + middleware order; controllers handle req/res; services hold the business logic. Easier to read, test, and reuse.

Q: What does `matchedData(req)` do and why use it?

Returns only the fields that passed validation, already sanitized/converted. Safer than reading raw `req.body` because unvalidated/extra fields are ignored.

Q: Why must the error handler be registered last?

Express runs middleware in order. The error handler (4 args) only catches errors from routes registered before it, so it goes at the very bottom.

Q: What's the difference between `requireUser` and `requireAdmin`?

`requireUser` blocks anonymous users (must be logged in). `requireAdmin` additionally requires the `admin` role. Both return 403 if not allowed.

B3. MongoDB / Mongoose

Q: What is the difference between a schema and a model?

A schema defines the shape/rules of a document. A model is built from the schema and is what you actually use to query and create documents (e.g. `Comment`).

Q: What does `populate()` do?

Replaces a stored reference (an `ObjectId` in a `ref` field) with the full referenced document — like a join. E.g. turn a comment's `userId` into the whole user object.

Q: How do you do pagination with Mongoose?

`.skip((page - 1) * limit).limit(limit)` to get one page, plus `countDocuments()` for the total so the frontend knows how many pages exist.

Q: What is a pre hook?

Code that runs before an event like save/validate. We use `pre("validate")` to auto-generate the public numeric ID (e.g. `commentId`) before the document is saved.

B4. Authentication & Security

Q: What is the difference between an access token and a refresh token?

Access token is short-lived (1h), sent with each request to prove you're logged in. Refresh token is long-lived (30d) and only used to get a new access token without logging in again.

Q: Why store tokens in HTTP-only cookies instead of `localStorage`?

HTTP-only cookies can't be read by JavaScript, so a cross-site scripting (XSS) attack can't steal the token.

Q: How are passwords stored?

Never in plain text. They're hashed with `bcrypt` plus a unique random salt per user. We compare a login attempt by hashing it the same way.

Q: What is a salt and why use one?

Random data added to a password before hashing so two users with the same password get different hashes — defeats precomputed (rainbow table) attacks.

Q: What are the two security incidents the dashboard tracks?

(1) Rate-limit hits — an IP making more than the allowed requests/min. (2) IP mismatch — the request IP differs from the IP stored in the JWT, suggesting a stolen token.

Q: Why can't anonymous users play games?

Playing wagers coins and affects ELO, so it requires an account. Anonymous users can still spectate. Enforced by `requireUser` on the backend and checks/redirects on the frontend.

B5. Real-time & Game Logic

Q: Why are dice rolls generated on the backend, not the frontend?

So players can't cheat. The server is authoritative — it knows all rolls and holds, and only reveals them at the end of the round. The frontend just sends which dice are held.

Q: Why WebSockets instead of normal HTTP requests for the game?

WebSockets keep a persistent two-way connection, so the server can push other players' actions instantly (low latency). Polling with HTTP would be slower and heavier.

Q: How is ELO adapted for more than two players?

Run the normal pairwise ELO calculation for every pair of players: you "win" against anyone who finished with fewer points and "lose" against anyone with more, then sum the changes.

Q: How does a tournament round work?

Players are randomly paired into games. When all games in the round finish, the next round pairs them again. After the final round, the winner is whoever has the most points; they get a trophy and bonus coins.

Part 4 – Cram Sheet (read this the morning of)

The highest-yield facts on 1–2 pages. If you only review one thing, review this.

The request flow (backend)

```
Request → CORS → Rate Limiter → setUserRole (JWT decode + IP check)
      → Route → Validator → validate → Controller → Service → DB → Response
```

- **errorHandler** is registered **last** so it catches everything.
- **Order on a route:** role check → validator → `validate` → controller.

Auth & tokens (most-asked topic)

- **Access token** = short (1h), sent every request. **Refresh token** = long (30d), gets a new access token.
- Both in **HTTP-only cookies** → JavaScript can't read them → safe from XSS.
- The user's **IP is stored inside the JWT**; if a request comes from a different IP → 401 + logged incident.

- Passwords: **script + a unique random salt** per user. Never stored in plain text.
- New users must **verify email** (token expires in **15 min**) before playing.
- On 401, the frontend auto-calls `/users/refresh` once, then retries.
- `req.mongoId` / `req.userRole` come from `setUserRole` after verifying the token — **never trust a userId from the request body**.

Roles

- **anonymous** → can spectate, can't play. **user** → can play. **admin** → everything.
- `requireUser` blocks anonymous (403). `requireAdmin` blocks non-admins (403).

The game (Part 5 / WebSockets — the hard one)

- Uses a **WebSocket** (persistent 2-way connection) for low latency, not HTTP.
- **Phases:** waiting → rolling → betting → round-end (3s reveal) → game-end.
- **The backend is authoritative:** it generates the dice rolls and decides the winner so nobody can cheat. The frontend only sends *which dice are held*.
- Game state is **in-memory** per match (lost on server restart); only the final result is saved via `recordMatch`.
- Each player has a **total time budget** (10/30/90s). Out of time → auto-roll + auto-match. Disconnect → auto-fold.
- **Incoming messages:** `join`, `ready`, `hold`, `done-rolling`, `bet`, `join-tournament`.
- Dice board is a **Web Component** (`<dice-poker-board>`), wrapped in React. It sends data back to React with `CustomEvent` + `dispatchEvent`.

ELO & coins

- Everyone starts at **1000 ELO**; separate rating per time control (10/30/90s).
- **Multiplayer ELO:** run pairwise — you "beat" everyone with fewer final points and "lose" to everyone with more, then sum ($K=32$).
- **Coins:** 100/week (granted lazily when profile is fetched), wagered on games (winner takes pot), **+500** for winning a tournament. Need enough coins for the buy-in.

Tournaments

- Random-pairing, round-based. Each round pairs players → creates Match objects.
- When all matches in a round finish → next round. After the last round → winner by points → trophy + bonus.
- Join only **before** start; leave anytime. Players auto-redirected into their game and back.

Security incidents (admin dashboard tracks 2)

1. **Rate-limit hit** — an IP exceeding **200 req/min** → 429, logs IP + user agent + time.
2. **IP mismatch** — request IP $\neq$ IP in the JWT → 401 (possible stolen token).

React essentials

- **useState** = state; changing it re-renders. **useEffect** = side effects; `[]` = once on mount, `[id]` = when id changes; the returned function = cleanup.
- **Context** (`useAuth` , `useAppearance`) shares state app-wide without prop drilling.
- **Keys** in `.map()` = unique IDs so React tracks list changes.
- **React.lazy + Suspense** in App.jsx = code splitting (load pages on demand).
- Custom hooks: `usePolling` (timer), `useFetch` (data), `useDebounceValue` (search).

Mongoose essentials

- **Schema** = shape/rules; **Model** = what you query with. `ref` + `populate()` = join.
- **Pagination**: `.skip((page-1)*limit).limit(limit) + countDocuments()`.
- `pre('validate')` hook auto-generates the public numeric IDs.

8 likely "gotcha" questions (say these out loud)

1. *Why HTTP-only cookies and not localStorage?* → XSS can't read them.
2. *What stops me posting as another user?* → `userId` comes from the verified JWT (`req.mongooseId`), not the body.
3. *Why generate rolls on the backend?* → server-authoritative; prevents cheating/bluffing.
4. *Why must errorHandler be last?* → middleware runs in order; it only catches what's above it.
5. *Why WebSockets over polling for the game?* → instant push, low latency, lighter.
6. *What's a salt for?* → identical passwords get different hashes; defeats rainbow tables.
7. *How does ELO work with 3–5 players?* → pairwise wins/losses summed.
8. *What happens if a player runs out of time?* → dice auto-rolled, they auto-match, can't hold.

Part 5 – Deep Dive (from Oblig 3 feedback)

The Oblig 3 examiner noted two advanced topics to review: **React's render/reconciliation algorithm and the role of `key`**, and **cleanup functions in `useEffect`**. These are the most likely places to get pushed in the defense, so know them properly — not just the one-liner.

5.1 Render & reconciliation (and where `key` fits)

The Virtual DOM. Your JSX doesn't create real DOM nodes directly. It produces a lightweight JavaScript tree of "element" objects (this is the **Virtual DOM**) describing what the UI *should* look like. Real DOM operations are slow, so React works on this cheap copy first.

What a re-render actually is. When a component's **state or props change**, React **re-runs that component function**, which produces a *new* Virtual DOM tree. Re-rendering is just "run the function again

to get the new tree" — it does **not** immediately touch the real DOM.

Reconciliation = the diffing step. React then compares the new tree against the previous tree and works out the **minimum set of real DOM changes** needed to go from old to new. That comparison is called **reconciliation**. It then applies (commits) only those minimal changes to the real DOM. So the flow is:

```
state/props change → re-run component → new Virtual DOM
                        → diff against old tree (reconciliation)
                        → apply minimal real-DOM updates (commit)
```

The two main heuristics React uses to keep diffing fast ($O(n)$ instead of $O(n^3)$): 1. **Different element type → throw the subtree away and rebuild it.** If a `<div>` becomes a `<span>` (or `ComponentA` becomes `ComponentB`), React doesn't try to diff inside — it unmounts the old one (running cleanup) and mounts the new one fresh. 2. **Same type → keep the node, update only what changed.** Same element type means React keeps the existing DOM node and just patches the attributes/children that differ.

Where `key` comes in — lists. For a list of siblings (a `.map()`), React needs to match each *new* child to the *old* child it corresponds to. By default it matches **by position (index)**. That's fine if the list never reorders, but it breaks for inserts/deletes/reorders. A **key** gives each element a **stable identity** so React can match "this is the same item as before" even if it moved — and then it moves the DOM node instead of rebuilding it and can preserve that item's state.

The classic pitfall (the part that gets you marked down): using the array index as the key. If you insert an item at the **front** of the list, every item's index shifts by one. React, matching by index, now thinks *every* item changed — so it does unnecessary work, and worse, **per-element state gets attached to the wrong item** (e.g. text typed into the 1st input suddenly appears on the 2nd). That's exactly why the project uses **stable unique IDs** as keys (`key={match.matchId}` , `key={tournament._id}`), never the index.

One-sentence version: "Re-rendering re-runs the component to build a new Virtual DOM; reconciliation diffs it against the old one and applies only the minimal real-DOM changes; keys give list items a stable identity so React matches them across renders instead of mismatching by index."

Likely follow-ups: - *Does a re-render always update the DOM?* No — only if reconciliation finds a difference. A re-render with an identical result commits nothing. - *Why not just use the index as a key?* It's only safe if the list never reorders/inserts/deletes; otherwise React misidentifies items, wasting work and corrupting per-item state. - *What happens to a component whose type changes?* It's unmounted (cleanup runs) and a brand-new one is mounted — its state is lost.

5.2 Cleanup functions in `useEffect`

What it is. The function you **return** from a `useEffect`. It's React's hook for "undo whatever this effect set up."

When it runs — exactly. It runs (1) right **before the effect runs again** (when a dependency changed), and (2) once when the component **unmounts**. So for an effect with deps `[id]`: mount → run effect; `id` changes → **cleanup of the old effect**, then run effect again; unmount → final cleanup.

Why it matters (what breaks without it): - **Memory leaks / duplicate work** — an interval or event listener that's never cleared keeps running (and a *new* one gets added on every re-run), so you stack up timers/listeners. - **Open connections** — a WebSocket that's never closed leaks a connection and keeps receiving messages for a page you've left. - **State updates after unmount / stale closures** — a fetch that resolves after the component is gone tries to `setState` on something that no longer exists. Aborting it (or guarding with a flag) avoids that.

Where the project actually does this: - **usePolling** — sets up `setInterval` and an `AbortController`; its cleanup does `clearInterval(timer)` and `controller.abort()`, so the timer stops and any in-flight request is cancelled when the interval restarts or the component unmounts. - **Game.jsx** — opens a WebSocket on mount; cleanup **closes** it on unmount so you're not still connected to a game room you left. - **Appearance.jsx** — adds a `mousedown` "click-outside" listener; cleanup `removeEventListener`s it so closed panels don't leave dangling listeners. - **useFetch** — passes an `AbortController` signal and aborts on cleanup so a slow response can't set state after unmount.

One-sentence version: "The cleanup function is what I return from `useEffect`; React runs it before the next effect run and on unmount, and I use it to clear intervals, remove listeners, close the WebSocket, and abort fetches so I don't leak timers/connections or call `setState` after the component is gone."

Likely follow-ups: - *When does cleanup run for `[]` deps?* Only on unmount (the effect never re-runs, so there's no "before next run"). - *What if you don't return a cleanup from an effect that adds a listener every render?* You stack a new listener on each run — a leak and duplicate handlers firing. - *Effect with no dependency array?* It runs after every render, and its cleanup runs before each subsequent run — usually a sign the deps array is missing.

Good luck on the exam!